

Encapsulation in Java

Encapsulation is the process of wrapping data (variables) and methods (functions) together inside a single unit (class) and restricting direct access to the data.

It is one of the four main principles of **Object-Oriented Programming (OOP)**.

- **Why Encapsulation is Important**
 - Protects data from unauthorized access
 - Improves security
 - Makes code more maintainable
 - Allows controlled modification of data
-

- **Encapsulation in the `Car` Class Example**

Step 1: Make variables private

To achieve proper encapsulation, fields should be declared `private`.

```
private int wheel;  
private String brand;
```

This prevents direct access like:

```
car.wheel = 5; // ❌ Not allowed if  
private
```

Step 2: Provide Public Getters and Setters

We use **getter** and **setter** methods to access and modify private data safely.

Getter (Read Data)

```
public int getWheel() {  
    return wheel;  
}
```

```
public String getBrand() {  
    return brand;  
}
```

Setter (Modify Data)

```
public void setWheel(int wheel) {  
    this.wheel = wheel;  
}
```

```
public void setBrand(String brand) {  
    this.brand = brand;  
}
```

- **Fully Encapsulated Version of Your Car Class**

```
public class Car {  
  
    private int wheel;  
    private String brand;  
  
    public Car(int wheel, String  
brand) {  
        this.wheel = wheel;  
        this.brand = brand;  
    }  
  
    public int getWheel() {  
        return wheel;  
    }  
}
```

```
    public void setWheel(int wheel) {
        this.wheel = wheel;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String
brand) {
        this.brand = brand;
    }

    public void show() {
        System.out.println("Brand: "
+ brand + ", Wheels: " + wheel);
    }
}
```

Real-World Meaning in This Example

1. `wheel` and `brand` are hidden (private).
 2. Users must use `getWheel()`, `setWheel()`, `getBrand()`, `setBrand()` to access or change values.
 3. This protects the internal state of the object.
-

Extra Improvement (Validation Example)

Encapsulation also allows validation:

```
public void setWheel(int wheel) {  
    if (wheel > 0) {  
        this.wheel = wheel;  
    }  
}
```

Now invalid values (like negative wheels) cannot be set.

Summary

1. Hides data
2. Provides controlled access
3. Improves security
4. Makes code flexible and maintainable

```
public class Car {
    int wheel;
    String brand;
}

// Constructor
public Car(int wheel, String brand) {
    this.wheel = wheel;
    this.brand = brand;
}

// Getter for wheel
public int getWheel() {
    return wheel;
}

// Setter for wheel
public void setWheel(int wheel) {
    this.wheel = wheel;
}

// Getter for brand
public String getBrand() {
    return brand;
}

// Setter for brand
public void setBrand(String brand) {
    this.brand = brand;
}

// Renamed method
public void show() {
    System.out.println("Brand: " + brand + ", Wheels: +
}
}
```